Jaypee Institute of Information Technology, Noida

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING AND INFORMATION TECHNOLOGY



Project Title: Klaro: A Private, Offline, Multilingual Assistant for Reading Tender Documents (RFPs)

Enrol. No.	**Name of Student**
9923103059	Yasharth Singh
9923103069	Nikunj Sethi
9923103072	Shrey Sircar

Course Name: Major Project Part-1

Course Code: 15B19CI791

Program: B. Tech. CSE

4th Year 7th Sem

2026 - 2027

ACKNOWLEDGEMENT

We would like to place on record our deep sense of gratitude to Prof. Shikha Mehta, HOD-CSE, Jaypee Institute of Information Technology, India, for her generous guidance, help and useful suggestions given to us throughout this project.

We express our sincere gratitude to her for the stimulating guidance, continuous encouragement and supervision throughout the course of the present work.

We also wish to extend our thanks to our classmates for their insightful comments and constructive suggestions to improve the quality of this project work.

Signature(s) of Students

Yasharth Singh	Nikunj Sethi	Shrey Sircar
(9923103059)	(9923103069)	(9923103072)

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and beliefs, it contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma from a university or other institute of higher learning, except where due acknowledgment has been made in the text.

Place: Noida
Date: _____

Name: Yasharth Singh
Enrolment No.: 9923103059

Name: Nikunj Sethi
Enrolment No.: 9923103069

Name: Shrey Sircar
Enrolment No.: 9923103072

CERTIFICATE

This is to certify that the work titled "Klaro: A Private, Offline, Multilingual Assistant for Reading Tender Documents (RFPs)" submitted by Yasharth Singh, Nikunj Sethi and Shrey Sircar of B.Tech of Jaypee Institute of Information Technology, Noida has been carried out under my supervision. This work has not been submitted partially or wholly to any other University or Institute for the award of any other degree or diploma.

_____

Signature of Supervisor

Name of Supervisor: Prof. Shikha Mehta
Designation: HOD-CSE
Date: _____

ABSTRACT

When a company wants to bid for a government or corporate contract, it first has to read the tender document, called a Request for Proposal (RFP). These documents often run to 50–100 pages, and the details that matter (who is asking, what work is wanted, who is allowed to bid, deadlines, payment terms, penalties) are scattered across them. RFPs are also confidential, so uploading them to an online AI service such as a public chatbot is often not allowed.

Klaro is a desktop application that reads an RFP and writes a short, organized summary of it entirely on the user's own laptop. Nothing is sent over the internet. It uses a small AI language model (Qwen3.5 with 4 billion parameters) that runs locally. Klaro breaks the RFP into manageable parts, asks the model two focused sets of questions about each part, combines the answers, and produces a summary in nine fixed headings. Every point in the summary shows the page number it came from, and clicking that number opens the original PDF at that page, so the reader can check it.

We compared three small AI models on the same test before choosing one, and tested the finished system on two real RFPs of 54 and 84 pages. A 54–84 page RFP takes about 22–25 minutes on an ordinary laptop. The system made up no information in any run, and every point carried a page number. We also found a weakness: for long documents the final step copies the collected facts instead of shortening them, and we explain how we plan to fix it. The report ends with the plan for the next phase, a Proposal Assistant with more AI helpers and search over past proposals.

Keywords: tender documents, RFP, offline AI, small language model, document summary, privacy.

Table of Contents

Page No.

Acknowledgement . . . . . *i*
Declaration . . . . . *ii*
Certificate . . . . . *iii*
Abstract . . . . . *iv*
List of Tables . . . . . *vii*
List of Figures . . . . . *viii*
List of Abbreviations . . . . . *ix*
List of Nomenclature . . . . . *x*

Chapter 1: INTRODUCTION

1.1 General 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Present Work 1

Chapter 2: BACKGROUND STUDY

2.1 General 2
2.2 Earlier Work on Tender Documents 2
2.3 Using a Little Human Help 2
2.4 Reading Very Long Documents 2
2.5 Small AI Models and Made-Up Answers 2
2.6 Summary of Gaps 3

Chapter 3: REQUIREMENT ANALYSIS

3.1 Who Uses Klaro 4
3.2 What the System Must Do 4
3.3 Quality Requirements 4
3.4 Hardware and Software Requirements 4

Chapter 4: DETAILED DESIGN

4.1 The Complete Planned System 5
4.2 What We Built in Phase 1 5
4.3 Plan versus What Was Built 5
4.4 The Document Processing Pipeline 6
4.5 Handling Long Documents 6
4.6 What the Summary Contains 7
4.7 Keeping the Document Safe 7

Chapter 5: IMPLEMENTATION

5.1 Tools Used 8
5.2 Reading and Splitting the PDF 8
5.3 Asking the AI Two Focused Questions 8
5.4 Combining Facts and Finding the Issuer 8
5.5 Writing and Checking the Summary 8
5.6 The Desktop Application 9
5.7 Testing 10

Chapter 6: EXPERIMENTAL RESULTS AND ANALYSIS

6.1 Choosing the AI Model 11
6.2 First Experiments and the Pilot Run 12
6.3 Full Runs through the Desktop App 12
6.4 What the Results Show 13

Chapter 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion 15
7.2 Limitations 15
7.3 Future Scope 15

References **17**

List of Tables

Table	Title	Page
2.1	What earlier work did and what it did not cover	3
3.1	Functional requirements	4
3.2	Hardware and software requirements	4
4.1	Plan versus what was built in Phase 1	6
4.2	The nine headings of the summary	7
5.1	Main tools and what each one does	8
6.1	Model benchmark: accuracy (47 hand-marked requirements)	11
6.2	Model benchmark: size, speed and memory	11
6.3	Model benchmark: following the answer format	11
6.4	Results of the full runs (Apple M4 laptop)	13

List of Figures

Figure	Title	Page
1.1	Klaro in one picture	1
4.1	The complete planned system	5
4.2	What was built in Phase 1	5
4.3	The document processing pipeline	6
4.4	How a long RFP is split, read in parts and put back together	7
5.1	What the user does in Klaro	9
5.2	Home screen: add an RFP, see past analyses and whether the AI model is ready	9
5.3	Progress screen while a 54-page RFP is being analyzed	10
6.1	Main benchmark scores of the three models	12
6.2	Results screen: summary with page numbers beside the original PDF	14
6.3	Results screen with the automatic quality checks	14
7.1	The planned Proposal Assistant	15

List of Abbreviations

4B	4 billion parameters (a measure of an AI model's size)
AI	Artificial Intelligence
CMMI	Capability Maturity Model Integration (a software-quality certificate)
GB	Gigabyte
LoRA	Low-Rank Adaptation (a low-cost way to further train a model)
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
RFP	Request for Proposal (a tender document)
TF-IDF	Term Frequency–Inverse Document Frequency (a keyword-scoring method)

List of Nomenclature

Agent	An AI helper with one job, for example writing the summary
Part (chunk)	A piece of the RFP of about 1,800 words that the AI reads at one time
Fact	One short statement taken from the RFP, with its page number
Executive summary	The short overview of the RFP under nine fixed headings
Pipeline	The fixed sequence of steps every RFP goes through
Local model	An AI model that runs on the user's own computer, not on the internet
Parameter	One of the numbers inside an AI model; more parameters usually means a bigger, slower model
Word piece (token)	The small unit of text an AI model reads and writes, often part of a word
Backed by facts	A summary point that matches a fact found in the RFP
Made-up information	A statement from the AI that is not in the document (also called hallucination)
Precision	Of the items the AI listed, the share that were correct
Recall	Of the items that should have been listed, the share the AI found
Benchmark	A fair test that gives several models the same task and compares their scores

Chapter 1: INTRODUCTION

1.1 General

A Request for Proposal (RFP) is a document an organization publishes when it wants companies to bid for a piece of work. Before a bid team can decide whether to respond, someone has to read the whole RFP and note who issued it, what work is required, who can bid, the deadlines, payment terms, contract length, and special conditions such as penalties or confidentiality rules. The two government RFPs we worked with are 54 and 84 pages long, and these points are spread across many sections.

AI chatbots can summarize long documents, but that means uploading the document to someone else's servers. Bid teams usually cannot do that with confidential tenders. Klaro takes a different approach: the AI model runs on the user's own laptop, so the document never leaves it (Figure 1.1).

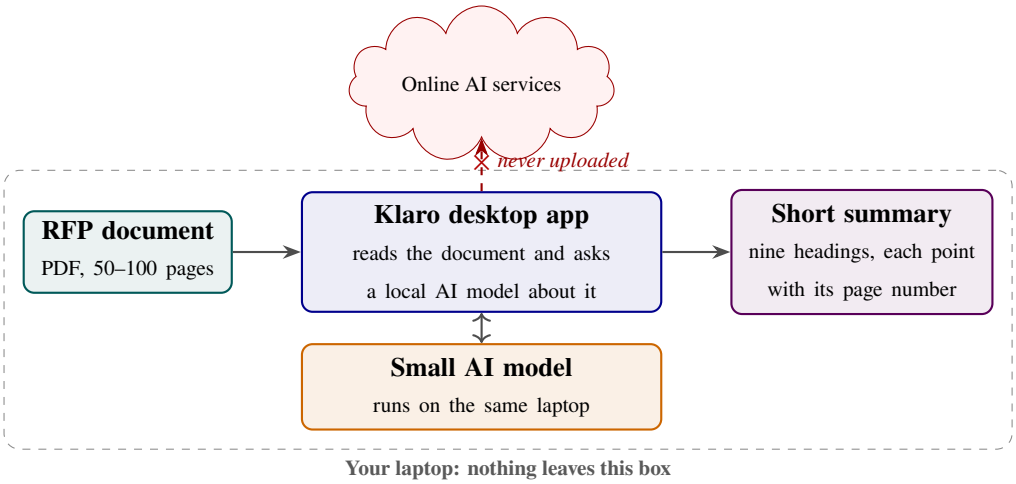


Figure 1.1: Klaro in one picture

1.2 Problem Statement

Given a tender document (RFP) as a PDF, produce a short summary under nine fixed headings, where every point shows which page it came from. It must run on an ordinary laptop, use only an AI model that runs on that laptop, and never send any part of the document over the internet. A non-technical reader must be able to check each point against the original, and when something goes wrong the app must say so clearly instead of showing a half-finished result.

1.3 Objectives

1. Build a step-by-step process that turns an RFP PDF into a page-referenced summary.
2. Pick the most suitable small AI model by testing several on the same task.
3. Measure the quality of the summaries and record the weak points honestly.
4. Package everything as a desktop app for Windows and macOS with progress updates and a side-by-side view of the original.

1.4 Scope of the Present Work

This phase handles English RFPs in text-based PDF form and one feature, the executive summary. German is the first planned language (mid-term); scanned files, Word files, requirement checklists, proposal drafts and search over past proposals also come later (Chapter 7).

Chapter 2: BACKGROUND STUDY

2.1 General

Computers have been used to analyze tender documents since before today's AI chatbots. Older methods counted important words, grouped similar sentences and picked out the most important ones. Newer methods use AI models that can read long text. We studied five papers close to our problem, plus background work on the kind of AI model we use.

2.2 Earlier Work on Tender Documents

Beason et al. [1] processed a large set of U.S. Department of Defense and Homeland Security RFPs to spot technology trends. They found that RFPs follow no common layout and contain a lot of noise (contents pages, boilerplate, legal text), which has to be cleaned out first. They also found that exact, structured facts (required technology, compliance items, timelines) are more useful to readers than a free-flowing summary. Klaro follows both points: it skips contents pages and produces fixed headings.

Trappey et al. [2] summarized more than 1,300 tenders for power transformers. They found key terms by counting words (TF-IDF), grouped sentences by topic, and picked the highest-ranked sentences with TextRank [3]. Keeping the top 35% of sentences preserved about 80% of the important requirements and cut reading time to about 29%. We took two ideas from this: summarize topic by topic rather than all at once, and aim for a real reduction in length.

2.3 Using a Little Human Help

Knisely and Pavliscsak [4] sorted research proposals into themes. A small number of proposals were labelled by experts, and those labels guided the computer in sorting the rest. This worked much better than sorting with no human input. We saw the same need in our results, where the AI sometimes filed a fact under the wrong heading, and it is why the next phase starts with a set of human-checked summaries.

2.4 Reading Very Long Documents

Standard AI models can only look at a limited amount of text at once, and the cost grows quickly as the text gets longer [5]. Longformer [6] reduces this cost so a model can read thousands of words in one go. Douzon et al. [7] went further for business documents by also using the page layout, so that a table value stays linked to its heading. Both show what is lost when a document is cut into pieces: links between distant sections. A small model on a laptop cannot read an 84-page RFP in one go, so Klaro does cut it into pieces, but it keeps page numbers on every piece and combines the results afterwards.

2.5 Small AI Models and Made-Up Answers

Language models sometimes state things that are not in the text they were given [8, 9], and they pay less attention to the middle of very long inputs [10]. One common remedy is to first look up the relevant passages and give only those to the model (retrieval-augmented generation, [11]). Klaro's remedy is to make the model cite a page for every point and then check each point against the collected facts. A model can later be specialized for a task with light-weight fine-tuning such as LoRA [12].

2.6 Summary of Gaps

Table 2.1: What earlier work did and what it did not cover

Work	What it did	What was missing for us
Beason et al. [1]	Found technology trends across many RFPs	No summary of a single RFP
Trappey et al. [2]	Picked key sentences from equipment tenders	Only copies sentences; one industry
Knisely & Pavliscsak [4]	Sorted proposals with a little expert help	Research proposals, not tenders
Longformer [6]	Let AI read very long text	Needs powerful hardware
Douzon et al. [7]	Used page layout for business documents	Too heavy for a laptop

None of these works on confidential tenders on an ordinary laptop, and none produces a summary where every point is linked to its page.

Chapter 3: REQUIREMENT ANALYSIS

3.1 Who Uses Klaro

The main user is a bid or proposal analyst who receives an RFP and has to brief a manager on it. This person is not expected to use programming tools. The project team also runs the same process from the command line to test changes. Figure 5.1 (Chapter 5) shows what the analyst does step by step.

3.2 What the System Must Do

Table 3.1: Functional requirements

No.	The system must…
R1	accept a text-based PDF and politely reject anything else;
R2	read the text page by page and remember the page numbers;
R3	split the document into parts small enough for the AI model;
R4	pull out facts from each part using the local AI model;
R5	combine the facts from all parts and remove repeats;
R6	write a summary under nine headings, with page numbers for every point;
R7	check the quality of the summary automatically;
R8	show progress while it works, and allow the user to cancel;
R9	show the summary beside the original PDF and jump to any cited page;
R10	keep a history of past analyses;
R11	check at start-up that the AI model is installed, and explain how to fix it if not.

3.3 Quality Requirements

- **Privacy:** no internet connection is used; everything, including fonts, is on the laptop.
- **Works on Windows and macOS:** the same app runs on both.
- **Honest about errors:** if a step fails, the app says which step and why.
- **Easy to change:** the AI model and folders are set in one settings file.
- **Repeatable:** every analysis is saved in its own folder with the model used and the time each step took.

3.4 Hardware and Software Requirements

Table 3.2: Hardware and software requirements

Item	Requirement (what we tested on)
Computer	At least 8 GB memory; tested on an Apple M4 laptop with 24 GB
Operating system	Windows 10/11 or macOS; tested on macOS 26.5
AI model runner	Ollama [13] with the `qwen3.5:4b` model [14] (about 3.4 GB download)
Analysis engine	Python 3.11 or newer, with pypdf [15], Pydantic [16] and FastAPI [17]
Desktop app	Electron [18] with React

Chapter 4: DETAILED DESIGN

4.1 The Complete Planned System

Figure 4.1 shows the full system we are building towards. The user works in a desktop app. The app talks to a local server, which hands work to a workflow engine. The engine runs three AI helpers ("agents"), each with one job, and everything is stored on the laptop.

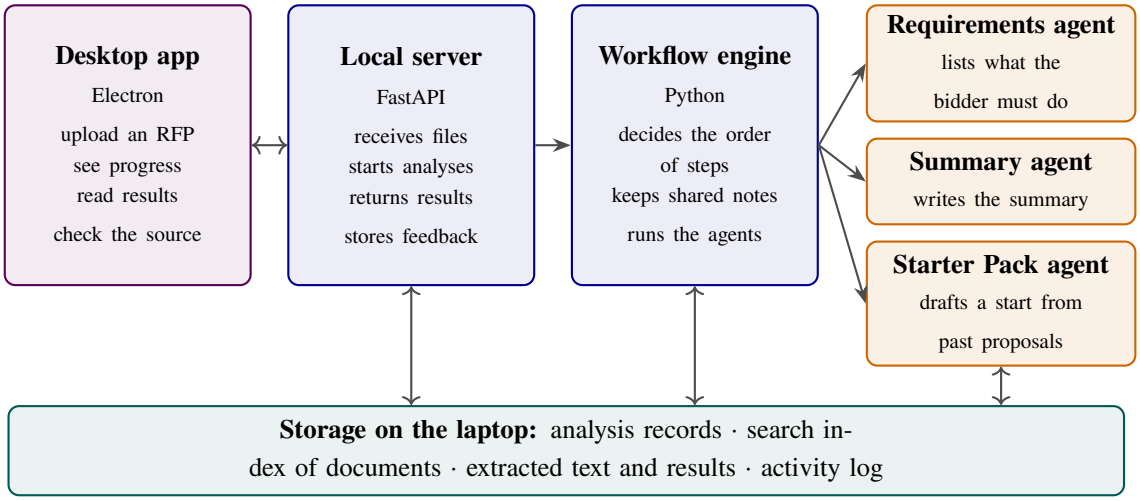


Figure 4.1: The complete planned system

4.2 What We Built in Phase 1

To get one feature working well before adding more, Phase 1 builds the summary agent only (Figure 4.2). When the app opens, it quietly starts the local server. When the user adds an RFP, the server starts the analysis as a separate background job, so the app stays responsive and the job can be cancelled at any time. Only one analysis runs at a time, because the laptop runs a single AI model; others wait in a queue. The job saves everything it produces in its own folder.

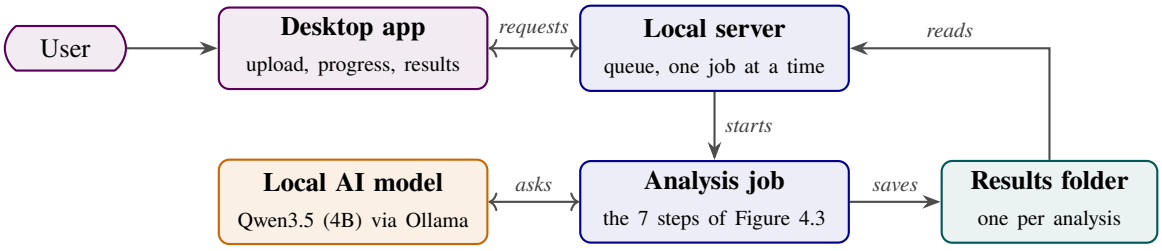


Figure 4.2: What was built in Phase 1

4.3 Plan versus What Was Built

The main ideas of the plan held: everything runs on the laptop, every point cites its page, the AI model can be swapped through one setting, and quality was measured before optimizing anything. Some parts were changed or postponed, as listed in Table 4.1.

Table 4.1: Plan versus what was built in Phase 1

Part	Planned	Built	Why
Desktop app	Tauri	Electron	Built-in PDF viewer for the side-by-side view; one programming language for the app
AI helpers	Three agents	Summary agent only	Get one right and measured first
Workflow engine	LangGraph [19]	A simple fixed sequence of 7 steps	One agent does not need a complex engine
Document search	Search index	Not needed yet	A summary reads the whole document anyway
Storage	Database	One folder per analysis	Easy to inspect; nothing to set up
Input files	PDF and Word	PDF only	Our test RFPs were PDFs
User feedback	Accept or correct each point	View and check only	Planned for Phase 2

4.4 The Document Processing Pipeline

Every analysis goes through the seven steps in Figure 4.3. Two of them use the AI model; the others are ordinary program logic, which is fast and always gives the same answer.

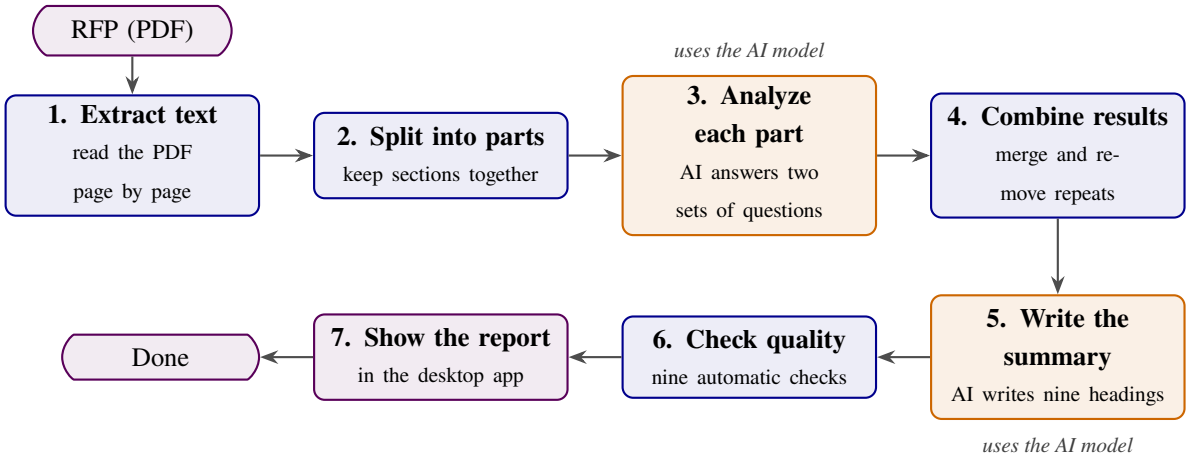


Figure 4.3: The document processing pipeline

4.5 Handling Long Documents

A small model cannot read a whole 84-page RFP at once. Klaro therefore works like a team splitting a long report: the document is cut into parts of about 1,800 words (roughly four pages), each part is read separately and its facts are noted with page numbers, and the notes from all parts are then combined into one list, which the summary agent turns into the final summary (Figure 4.4). Neighbouring parts overlap by 200 words so that a sentence cut at a boundary is not lost.

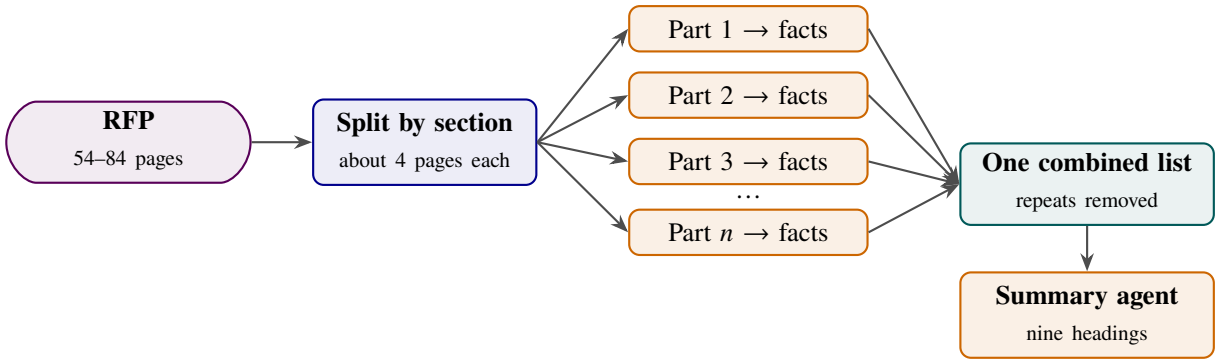


Figure 4.4: How a long RFP is split, read in parts and put back together

4.6 What the Summary Contains

Table 4.2: The nine headings of the summary

Heading	What goes under it
Overview	What is being bought and why, in two or three sentences
Issuing organization	Who published the RFP
Scope	The work and deliverables required
Key requirements	Documents and forms the bidder must submit
Eligibility	Who may bid: certificates, turnover, experience
Important dates	Deadlines, how long a bid stays valid, review dates
Commercial requirements	Pricing, billing, payment terms, taxes, deposits
Project duration	How long the contract runs, and extensions
Important conditions	Penalties, confidentiality, ownership of work, exit and termination rules

4.7 Keeping the Document Safe

The app only connects to programs on the same laptop: its own local server and the AI model runner. The screen part of the app is locked down so that it cannot read or write files by itself; it can only ask for three things (the server address, a file-picker dialog, and opening a web link in the browser). The PDF viewer is allowed to show only the RFP of the current analysis and nothing else, and the summary panel can only ask the app to jump to a page number that actually exists in the document.

Chapter 5: IMPLEMENTATION

5.1 Tools Used

Table 5.1: Main tools and what each one does

Tool	Its job in Klaro
Ollama [13]	Runs the AI model on the laptop
Qwen3.5 4B [14]	The AI model that reads the RFP parts and writes the summary
Python with pypdf [15]	Reads text out of the PDF, page by page
Pydantic [16]	Checks that the AI's answers have the expected structure
FastAPI [17]	The local server between the app and the analysis
Electron and React [18]	The desktop app the user sees

5.2 Reading and Splitting the PDF

Text is read from each page and stored with its page number. Contents pages near the start are skipped. Section headings are recognized only when they look like real headings, such as "4. SCOPE OF WORK" or "SECTION-D: …"; ordinary words in capital letters are ignored because RFPs contain many of them. Each part is then built from whole sections where possible.

5.3 Asking the AI Two Focused Questions

Early tests showed that asking the model for everything at once gave poor results: it missed items and its answers were sometimes cut off. So each part is read twice. The first pass asks about the overview, scope, requirements and eligibility. The second pass asks about dates, money, duration and legal conditions. Each answer must be in a fixed structured format with a page number for every point. If the model's answer is slightly malformed, Klaro recovers the usable points instead of throwing the whole answer away. If the model fails on more than half of the parts, the analysis is marked as failed rather than continuing with missing information.

5.4 Combining Facts and Finding the Issuer

The facts from all parts are grouped by heading, repeats are removed and their page numbers are combined. Klaro also works out which organization issued the RFP by looking for the organization name mentioned most often (names ending in words such as Bank, Authority or Ministry). The first version assumed a fixed organization whenever it found no match, so every RFP was credited to the organization of our first test document. The new version answers "Not identified" when it is not sure.

5.5 Writing and Checking the Summary

The combined facts are given to the model with instructions to write the nine headings, keep the page numbers, and use only the facts provided. The result is checked for the right structure. If a heading comes back empty although facts exist for it, those facts are copied in directly so that nothing is lost.

Klaro then checks the summary automatically without needing a human. The checks answer simple questions: Does every heading exist? Is every point backed by the facts found in the document? Does every point have a page number? How many of the facts made it into the summary? Are any points repeated? How much shorter is the summary than the facts? Is anything filed under the wrong heading? These checks compare words, so they show whether the summary agrees with what was extracted, not whether

the extraction itself was right.

5.6 The Desktop Application

When the app opens, it starts the local server in the background and waits until it is ready. If the AI model is missing, a setup screen explains how to install it. Figure 5.1 shows what the user does, and Figures 5.2 and 5.3 show two of the screens.

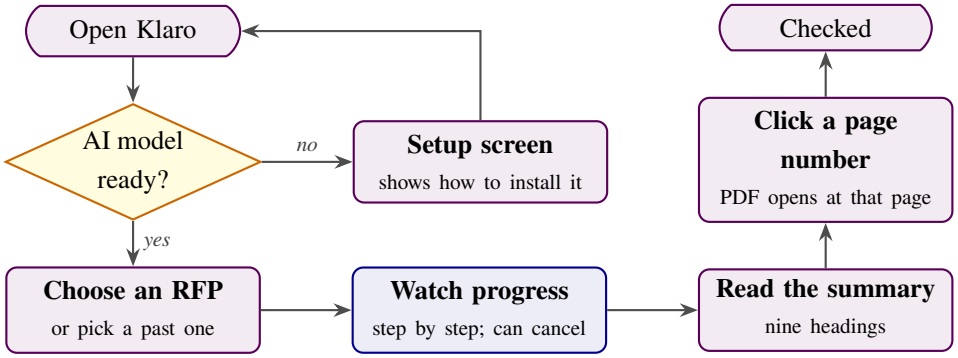


Figure 5.1: What the user does in Klaro

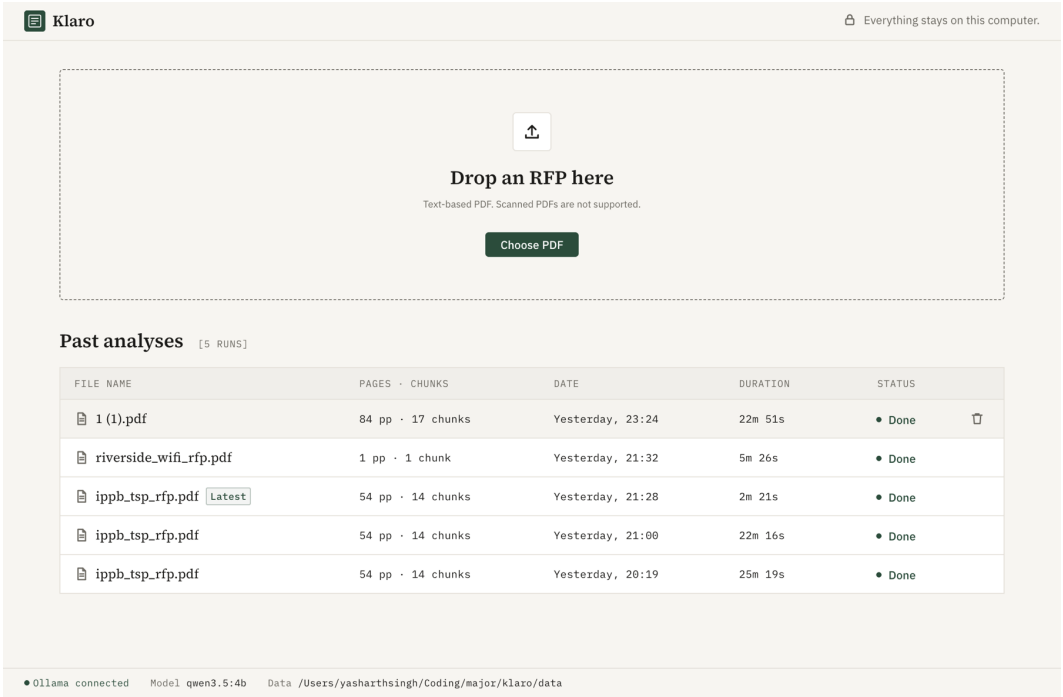


Figure 5.2: Home screen: add an RFP, see past analyses and whether the AI model is ready

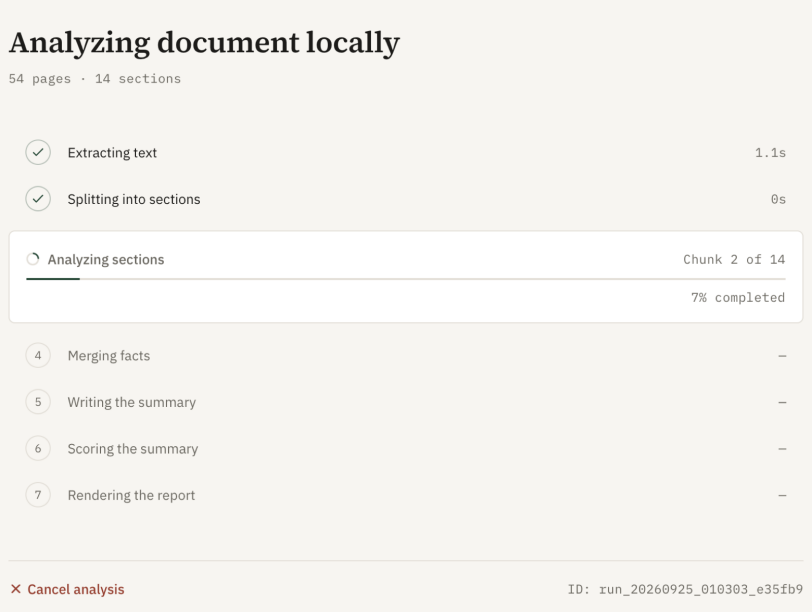


Figure 5.3: Progress screen while a 54-page RFP is being analyzed

5.7 Testing

Fifteen automatic tests check the report pages, the order of the pipeline steps, the handling of failed parts, and issuer detection. All fifteen pass. The AI steps were tested end to end on the documents in Chapter 6. The app was tested by hand on macOS; a teammate tests it on Windows.

Chapter 6: EXPERIMENTAL RESULTS AND ANALYSIS

6.1 Choosing the AI Model

We compared three small models that can run on a laptop: Gemma 4 (e4b), Qwen3.5 (4B) and Phi-4-mini. Each was given the same 20 RFP parts and asked to list the requirements in them. Team members had already marked 47 requirements in those parts by hand, and each model's list was compared with this answer sheet. A requirement counted as found (a match, at an 80% wording-similarity threshold) when the model's wording closely matched the hand-marked one, and standard information-retrieval metrics were computed from that: precision, recall and F1-score. All three models used the same instructions. Tables 6.1 to 6.3 break the results into accuracy, size and speed, and output format, and Figure 6.1 compares the main scores.

Table 6.1: Model benchmark: accuracy (47 hand-marked requirements)

	Gemma 4 (e4b)	**Qwen3.5 (4B)**	Phi-4-mini
Requirements found (out of 47)	33	23	7
Wrong or extra items listed	22	**8**	9
Precision (correct when it listed something)	60%	**74%**	44%
Recall (share of requirements found)	**70%**	49%	15%
F1-score (balance of precision and recall)	**65%**	59%	22%
Mandatory vs. optional judged correctly	85%	**91%**	57%
Made-up requirements	0%	0%	6%

Table 6.2: Model benchmark: size, speed and memory

	Gemma 4 (e4b)	Qwen3.5 (4B)	Phi-4-mini
Model size (billions of parameters)	8.0	4.7	3.8
Download size on disk (GB)	9.6	3.4	2.5
Typical time per part (seconds)	6.7	8.1	1.3
Slowest parts, 95th percentile (seconds)	15.9	22.7	18.7
Writing speed (word pieces per second)	28.5	19.0	48.5
Peak memory used (GB)	4.4	6.0	4.3

Table 6.3: Model benchmark: following the answer format

	Gemma 4 (e4b)	Qwen3.5 (4B)	Phi-4-mini
Answer usable after tolerant reading	100%	100%	100%
Answer in exactly the requested format	45%	0%	35%
Answer given as a bare list instead	0%	30%	65%

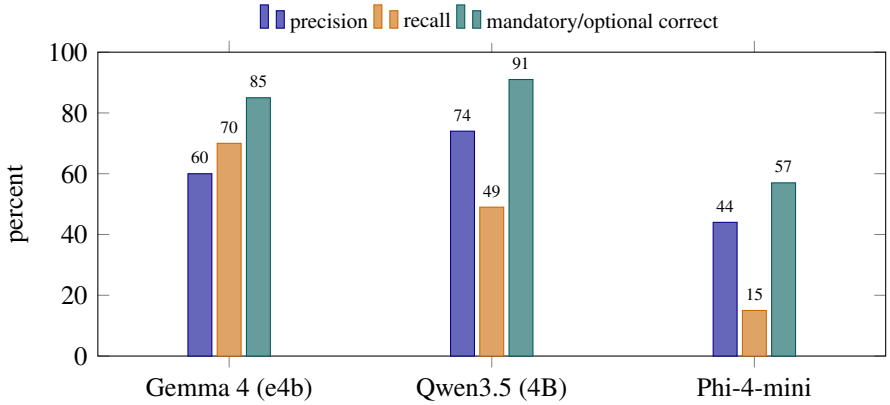


Figure 6.1: Main benchmark scores of the three models

Why we chose Qwen3.5. Phi-4-mini was fastest but found only 7 of 47 requirements and was the only model that made one up, so it was ruled out. Between the other two, Gemma found more requirements and was faster, which gave it the higher overall score, but it is a much bigger model: 8 billion parameters and a 9.6 GB download, almost three times Qwen's 3.4 GB (4.7 billion parameters). Every user has to download and store the model, so a smaller model makes Klaro practical on ordinary laptops with limited disk space. Qwen was the more careful model: when it listed something it was right 74% of the time against Gemma's 60%, it listed 8 wrong items against Gemma's 22, and it was best at telling mandatory from optional requirements. For a summary that people will rely on, a wrong point does more harm than a missing one, because a missing point can be found by asking more focused questions (which is why Klaro asks two sets of questions per part), while a wrong point may go unnoticed. Qwen's weakness in following the exact answer format is handled by Klaro's tolerant reading of answers. Because the model is chosen in one setting, Gemma can be swapped in for a comparison at any time; later tests with improved instructions raised Gemma's overall score to 67%, so it remains the main alternative.

6.2 First Experiments and the Pilot Run

Before the benchmark, the first attempt asked the model for all requirements in a part with a single question. On five parts, two answers were empty, one was cut off and unusable, and single answers took 2–10 minutes. This led to the two-question design. The first full run of that design on the 84-page RFP, by a teammate, gave a summary of 51 points; every point was backed by the extracted facts and had a page number, no information was made up, 76% of the facts were included, and the summary was 36% shorter than the facts. Nine points were under the wrong heading, for example a CMMI certificate listed under conditions instead of eligibility.

6.3 Full Runs through the Desktop App

We then ran the finished app on two RFPs across three runs: an 84-page government RFP for software development services (R1), and a 54-page bank RFP for a technical service provider, run twice (R2, R3). Table 6.4 shows the results.

Table 6.4: Results of the full runs (Apple M4 laptop)

	R1	R2	R3
Pages	84	54	54
Parts analyzed	17	14	14
Total time	22 min 51 s	25 min 19 s	22 min 16 s
Points in the summary	170	211	216
Points backed by facts / with page numbers	100% / 100%	100% / 100%	100% / 100%
Facts included in the summary	99%	97%	96%
Shorter than the facts by	8%	8%	9%
Made-up information	none	none	none
Parts where the AI failed	0	0	0

6.4 What the Results Show

6.4.1 Time Taken

About 95% of the time goes into step 3, where the AI reads each part twice: about 1.5 minutes per part. Writing the summary takes about one more minute. A 50–85 page RFP therefore takes 22–25 minutes, which is why the progress screen shows which part is being read and roughly how long is left.

6.4.2 Why Some Scores Look Too Good

When we compared the files, every point under the seven list headings was an exact copy of a collected fact: for long RFPs the model wrote only the overview, and the safety step of Section 5.5 filled in the rest. So the summary for a long RFP is currently the cleaned-up list of facts rather than a real summary, and the "100% backed by facts" score is high partly because of that copying. The most likely reason is that the list of facts for a long RFP is too long for the amount of text the model is set up to read at once. We have not confirmed this yet; it is the first fix planned for Phase 2.

6.4.3 Same Document, Different Answers

R2 and R3 were the same PDF, but the AI found 236 facts the first time and 244 the second, and the number of points under some headings differed by up to ten. The model does not give identical answers on every run, so improvements must be judged over several runs, not one.

6.4.4 Issuer Detection

With the original code, the bank RFP would have been credited to the government agency of our first test document. After the fix, the same saved facts gave "India Post Payments Bank Limited", with the ten pages where the name appears (Figure 6.2).

6.4.5 Checking the Summary

Every point shows its page numbers; clicking one opens the PDF at that page on the right (Figure 6.2). The Quality tab shows the automatic checks with a note that they compare words, not meaning (Figure 6.3). During all runs, the app made no internet connections.

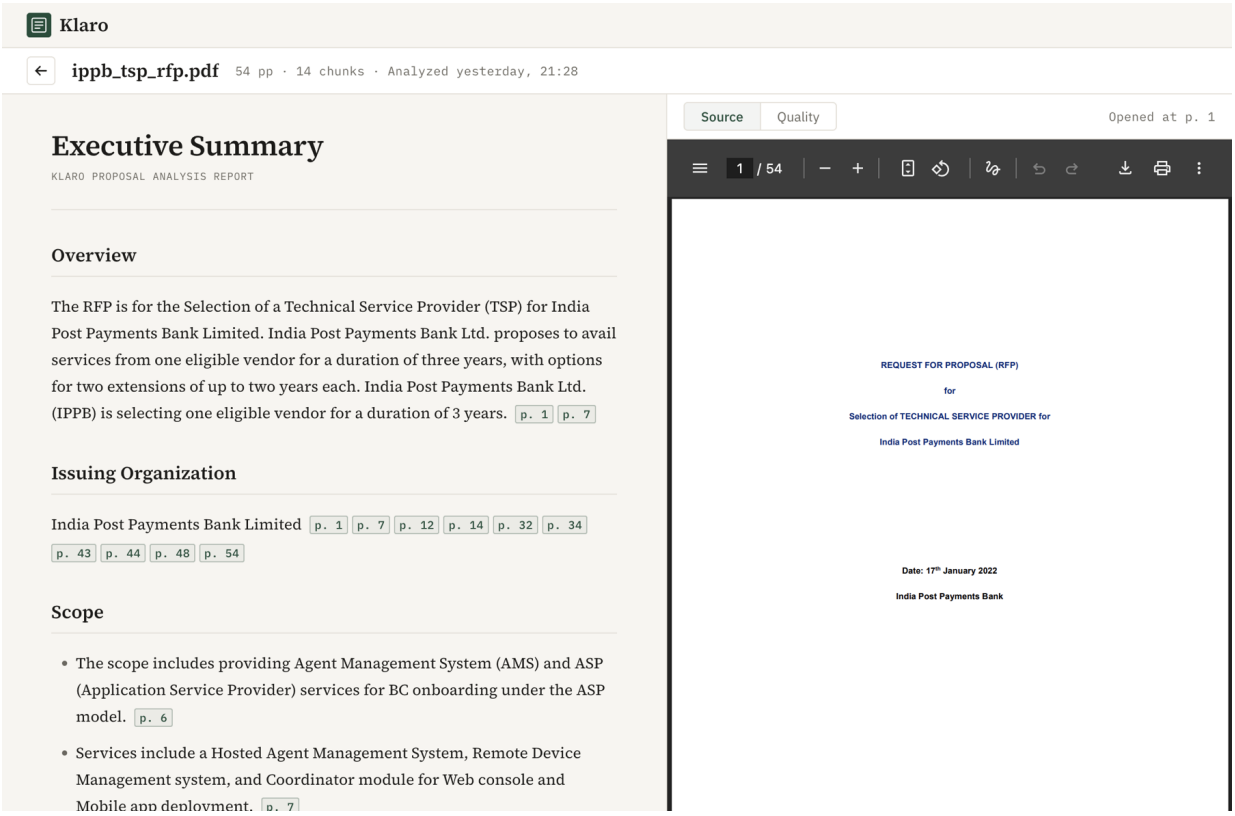


Figure 6.2: Results screen: summary with page numbers beside the original PDF

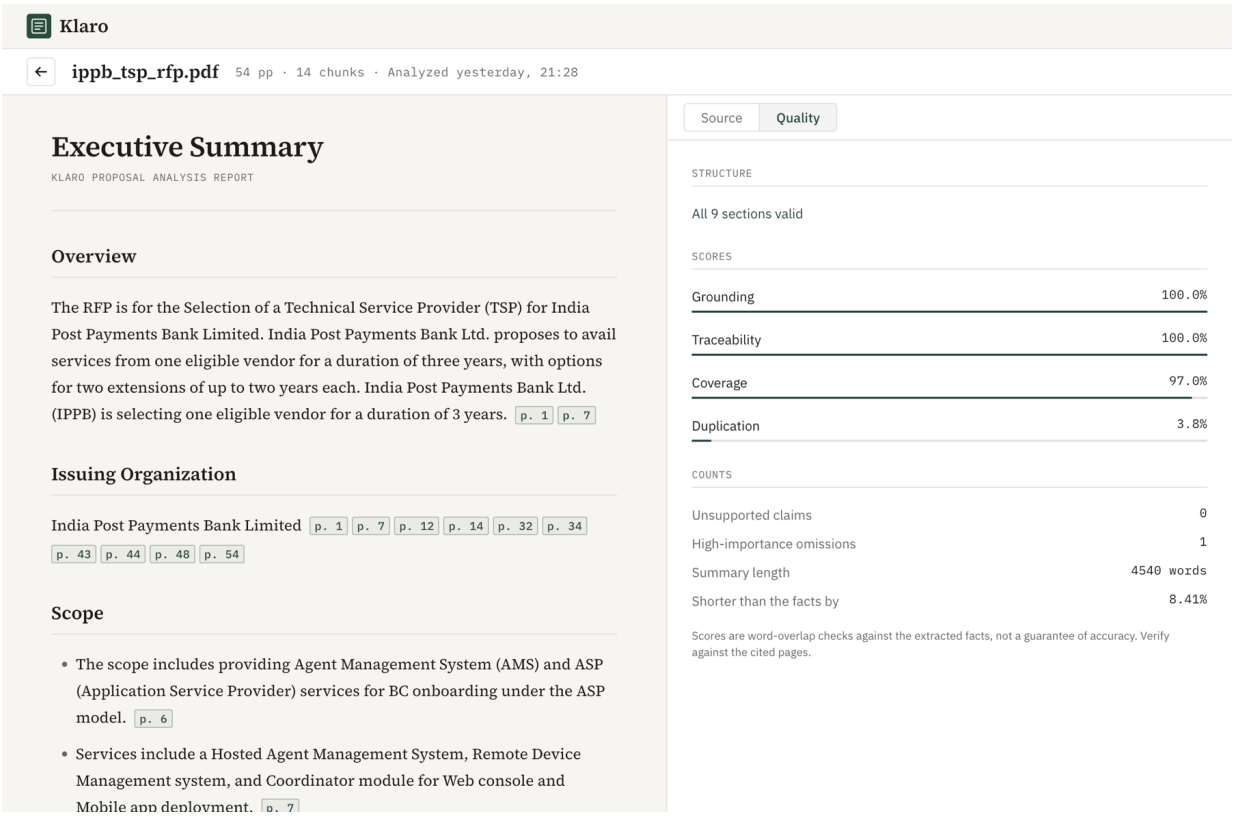


Figure 6.3: Results screen with the automatic quality checks

Chapter 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

Klaro's first phase works end to end: a 54–84 page tender goes in, and 22–25 minutes later a nine-heading summary comes out on an ordinary laptop, with a page number on every point and nothing sent over the internet. We chose the AI model by testing three on the same task, and chose Qwen3.5 because it made the fewest wrong claims and is about a third of the size of the closest competitor. The same process runs from the command line for testing and inside a desktop app for analysts, and when something goes wrong (the model is missing, the PDF is a scan, the AI fails on too many parts) the app says so clearly. Testing also showed where the system falls short, mainly that summaries of long RFPs are not yet shortened, and those findings set the order of work for the next phase.

7.2 Limitations

- Only three RFPs were tested, so we cannot yet say how well it works on tenders in general.
- There are no human-written reference summaries yet to compare against.
- Summaries of long RFPs are not yet shortened (Section 6.4.2).
- The AI can give different answers on different runs, and 22–25 minutes is slow.
- Only English, text-based PDFs work; tables lose their layout.
- There is no installer yet; the app runs from the project folder.

7.3 Future Scope

7.3.1 Proposal Assistant

Klaro will grow into a Proposal Assistant with two ways of working (Figure 7.1). In the **agentic workflow**, three AI helpers share the same reading of the RFP: the existing Executive Summary agent, a Requirements agent that produces a checklist of what the bidder must do, and a Starter Pack agent that turns the analysis into a first outline of the proposal, with risks and questions to ask the client. In the **file search workflow**, the company's past proposals and case studies are indexed on the laptop, and the ones most similar to a new RFP are found and handed to the Starter Pack agent.

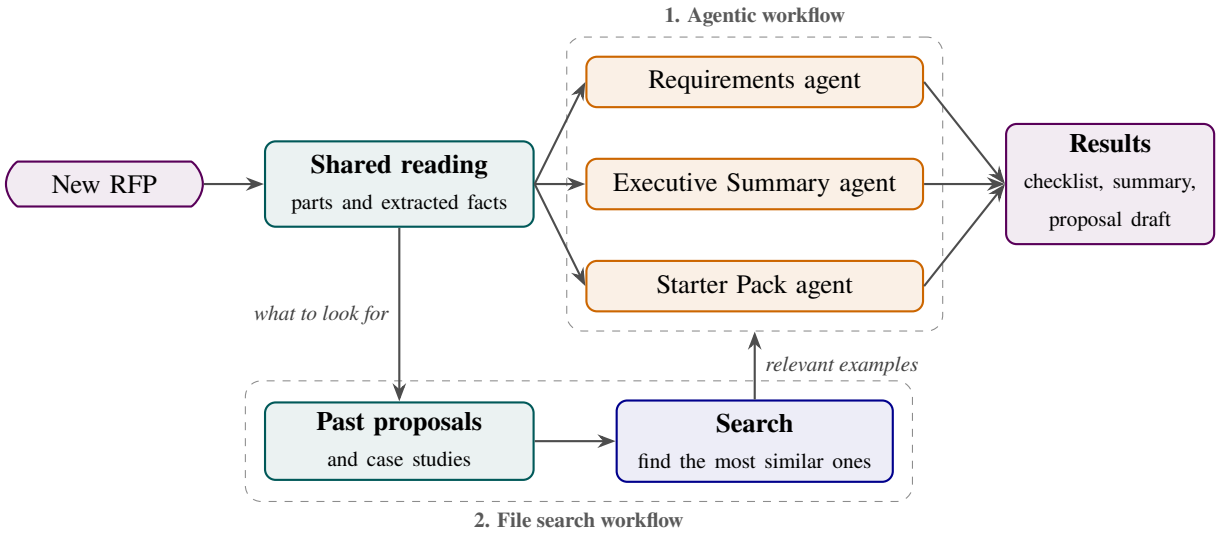


Figure 7.1: The planned Proposal Assistant

7.3.2 Real-Time Progress Updates

Today the app shows which step and which part it is working on. The next step is to show the results of each step as they are produced, so users do not wait for the final result: facts appear under their headings as each part is read, the summary appears as the model writes it, and the Starter Pack fills in section by section.

7.3.3 Better Summaries and Better Testing

- Fix the long-document problem: let the model read more at once, write one heading at a time, and record when the safety copy is used.
- Collect 30–40 real RFPs and build a reference set: a larger LLM drafts a first summary for each one, a human then corrects it, and the corrected version becomes the golden summary used to test every future change.
- Fine-tune Qwen3.5 with LoRA, a low-cost way to further train a model without retraining it in full, on these golden summaries, instead of relying only on prompting, and compare the fine-tuned model with the current one on the same benchmark. This targets the copying problem of Section 6.4.2: a model that has actually seen the summarization task, not just an instruction describing it, should need less safety-copy fallback and produce a genuinely shorter summary on long RFPs.
- Repeat the model benchmark on the summary task, with Gemma 4 as the main alternative.

7.3.4 App Improvements

- One-click installers for Windows and macOS, with guided setup of the AI model.
- Support for Word files, scanned PDFs and tables.
- Multilingual support, starting with German: read German RFPs and write the summary in German or English, since the multilingual extension was planned from the start.
- Let users accept, edit or flag each point, and use that feedback to measure quality.

REFERENCES

[1] S. Beason, W. Hinton, Y. A. Salamah, and J. Salsman, "Automated analysis of RFPs using natural language processing (NLP) for the technology domain," *SMU Data Science Review*, vol. 5, no. 1, 2021, art. 1.

[2] A. J. C. Trappey *et al.*, "Intelligent RFQ summarization using natural language processing, text mining, and machine learning techniques," *Journal of Global Information Management*, vol. 30, no. 1, pp. 1–26, 2022.

[3] R. Mihalcea and P. Tarau, "TextRank: Bringing order into text," in *Proc. Conf. Empirical Methods in Natural Language Processing*, 2004, pp. 404–411.

[4] B. M. Knisely and H. H. Pavliscsak, "Research proposal content extraction using natural language processing and semi-supervised clustering: A demonstration and comparative analysis," *Scientometrics*, vol. 128, no. 5, pp. 3197–3224, 2023.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.

[6] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," arXiv:2004.05150, 2020.

[7] T. Douzon, S. Duffner, C. Garcia, and J. Espinas, "Long-range transformer architectures for document understanding," in *Proc. ICDAR 2023 Workshops on Document Analysis and Recognition*, ser. Lecture Notes in Computer Science. Springer, 2023, pp. 47–64.

[8] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[9] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," in *Proc. 58th Annu. Meeting Association for Computational Linguistics*, 2020, pp. 1906–1919.

[10] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[11] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Advances in Neural Information Processing Systems 33*, 2020, pp. 9459–9474.

[12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[13] "Ollama: Get up and running with large language models locally," https://github.com/ollama/ollama, accessed: Sep. 24, 2026.

[14] "qwen3.5 model library entry," https://ollama.com/library/qwen3.5, accessed: Sep. 24, 2026.

[15] "pypdf documentation," https://pypdf.readthedocs.io, accessed: Sep. 24, 2026.

[16] "Pydantic documentation," https://docs.pydantic.dev, accessed: Sep. 24, 2026.

[17] "FastAPI documentation," https://fastapi.tiangolo.com, accessed: Sep. 24, 2026.

[18] "Electron documentation: Security," https://www.electronjs.org/docs/latest/tutorial/security, accessed: Sep. 24, 2026.

[19] "LangGraph documentation," https://langchain-ai.github.io/langgraph/, accessed: Sep. 24, 2026.